

Practice Set: Functions, Pointers, and Recursion

Instructions

This document contains 30 programming questions based on:

- Functions
- Pointers
- Recursion

Each question includes a descriptive problem statement and 2–3 test cases.

Questions

1. Sum of Digits using Recursion

Write a recursive function `sumDigits(int n)` that computes the sum of digits of an integer without using loops.

Test Cases:

Input: 1234 Output: 10

Input: 987 Output: 24

2. Greatest Common Divisor (GCD) Using Recursion

Using Euclid's recursive algorithm, write a function to return the GCD of two integers.

Test Cases:

Input: 24 36 Output: 12

Input: 17 5 Output: 1

3. Reverse a String using Pointers

Implement a function that reverses a string in-place using only pointers (no indexing allowed).

Test Cases:

Input: "hello" Output: "olleh"

Input: "abcd" Output: "dcba"

4. **Check Palindrome using Recursion**

Write a recursive function to check whether a string is palindrome. Do not use loops.

Test Cases:

Input: "racecar" Output: YES

Input: "hello" Output: NO

5. **Array Maximum using Recursion**

Given an array, find the maximum element using a recursive function.

Test Cases:

Input: [1,4,2,9,3] Output: 9

Input: [7,2] Output: 7

6. **Pointer Swap Function**

Write a function `swap(int *a, int *b)` that swaps two values using pointers.

Test Cases:

Input: a=5, b=9 Output: a=9, b=5

Input: a=-1, b=10 Output: a=10, b=-1

7. **Count Vowels in String using Recursion**

Implement a recursive function to count vowels in a string.

Test Cases:

Input: "education" Output: 5

Input: "sky" Output: 0

8. **Compute Power using Recursion**

Write a recursive function `power(a,b)` that computes a^b without loops.

Test Cases:

Input: 2 5 Output: 32

Input: 3 3 Output: 27

9. **Binary Search using Recursion**

Implement recursive binary search on a sorted array. Return index or -1 if not found.

Test Cases:

Input: [1,2,3,4,5], key=4 Output: 3

Input: [1,3,7], key=2 Output: -1

10. **Count Occurrences using Recursion**

Count how many times a key appears in an array through recursion.

Test Cases:

Input: [1,2,2,3], key=2 Output: 2

Input: [5,5,5], key=5 Output: 3

11. String Length using Pointers

Implement your own `strlen()` using pointer arithmetic.

Test Cases:

Input: "hello" Output: 5

Input: "AI" Output: 2

12. Fibonacci using Memoized Recursion

Use recursion + a pointer-based memo table to compute the n th Fibonacci number.

Test Cases:

Input: 10 Output: 55

Input: 1 Output: 1

13. Print Array in Reverse using Recursion

Print array elements in reverse order using recursion (without modifying array).

Test Cases:

Input: [1,2,3] Output: 3 2 1

Input: [4,9] Output: 9 4

14. Sum of Array using Pointers

Write a function to compute array sum using only pointer arithmetic.

Test Cases:

Input: [1,2,3] Output: 6

Input: [10,10] Output: 20

15. Check Sorted Array using Recursion

Determine if an array is strictly increasing using recursion.

Test Cases:

Input: [1,2,3,4] Output: YES

Input: [3,2,1] Output: NO

16. Find First Occurrence using Recursion

Return the first index of a key in array using recursive search.

Test Cases:

Input: [1,2,2,3], key=2 Output: 1

Input: [10,20], key=5 Output: -1

17. Multiply using Recursion

Implement multiplication using repeated addition recursively.

Test Cases:

Input: 5 3 Output: 15

Input: -4 2 Output: -8

18. Rotate Array Right by One using Pointers

Rotate array to the right by one position using pointer logic only.

Test Cases:

Input: [1,2,3,4] Output: [4,1,2,3]

Input: [10,20] Output: [20,10]

19. Check Prime using Recursion

Write a recursive function to check if a number is prime.

Test Cases:

Input: 7 Output: YES

Input: 12 Output: NO

20. String Copy using Pointers

Implement your own `strcpy()` using pointer dereferencing.

Test Cases:

Input: "hello" Output: "hello"

Input: "abc" Output: "abc"

21. Find Last Occurrence using Recursion

Return the last index of a key in array recursively.

Test Cases:

Input: [1,2,2,3], key=2 Output: 2

Input: [5,5,5], key=5 Output: 2

22. String to Uppercase using Pointers

Convert lowercase letters of a string to uppercase using pointer operations.

Test Cases:

Input: "hello" Output: "HELLO"

Input: "AI" Output: "AI"

23. Tower of Hanoi

Print the step-by-step sequence to move n disks from source to destination using recursion.

Test Cases:

Input: 2 Output: 3 moves

Input: 3 Output: 7 moves

24. Compute nCr using Recursion

Use the recursive formula: $nCr = (n-1)C(r-1) + (n-1)Cr$

Test Cases:

Input: 5 2 Output: 10

Input: 6 3 Output: 20

25. Find Minimum using Recursion

Compute the minimum element in an array recursively.

Test Cases:

Input: [9,3,6] Output: 3

Input: [10] Output: 10

26. Print Digits in Words using Recursion

Convert digits of a number to words recursively.

Test Cases:

Input: 45 Output: "four five"

Input: 908 Output: "nine zero eight"

27. Reverse Linked List using Recursion (Conceptual)

Explain and write recursive code to reverse a singly linked list.

Test Cases:

Input: $1 \rightarrow 2 \rightarrow 3$ Output: $3 \rightarrow 2 \rightarrow 1$

Input: $7 \rightarrow 8$ Output: $8 \rightarrow 7$

28. Length of Linked List using Recursion

Calculate number of nodes in a linked list using recursive traversal.

Test Cases:

Input: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ Output: 4

Input: (empty) Output: 0

29. Pointer-based Bubble Sort (Conceptual)

Sort an array using bubble sort implemented purely with pointer arithmetic.

Test Cases:

Input: [4,1,3] Output: [1,3,4]

Input: [10,5] Output: [5,10]

30. K-th Node from End using Recursion

Using recursion and backtracking, print the k -th node from the end of a linked list.

Test Cases:

Input: List: $2 \rightarrow 4 \rightarrow 6 \rightarrow 8$, $k=2$ Output: 6

Input: List: 1→2, k=1 Output: 2